

Lab 4 - Мікросервіси з використанням Messaging queue (10 балів)

Завдання базується на функціоналі, розробленому у попередньому завданні, і є його подальшим розвитком.

У зв'язку з тим, що **counter-service** взаємодіє з дисковою БД для оновлення значень на рахунку, така операція може бути повільною. Тому **facade-service** може не чекати на її завершення і відправляти запит на оновлення рахунку асинхронно через message queue.

До **counter-service** необхідно додати:

- у якості каналу доставки повідомлень між **facade-service** та **counter-service** використати message queue
- **facade-service** для читання даних використовує попередній спосіб - через GET-запит. Зміни торкаються лише POST-запиту

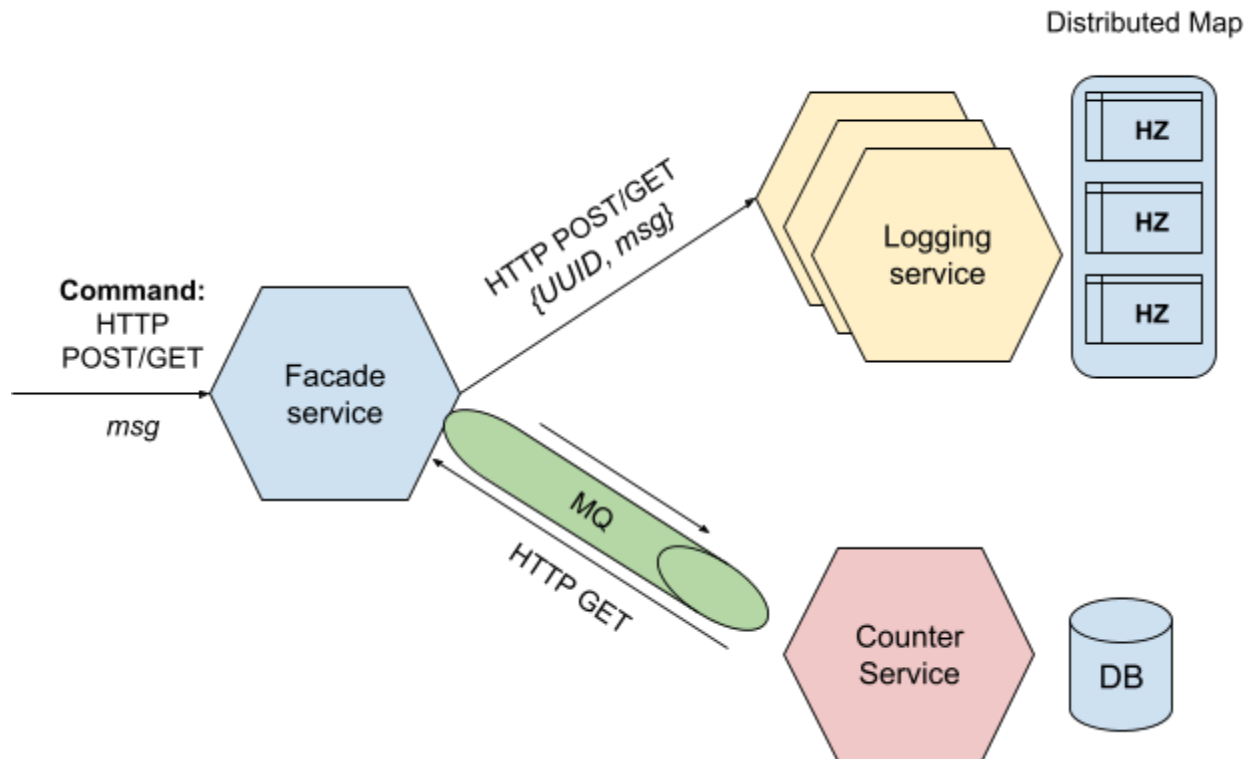
Пропонується використати у якості Messaging queue

- Hazelcast Queue <https://docs.hazelcast.com/hazelcast/5.5/data-structures/queue> або
- Kafka - <https://hub.docker.com/r/apache/kafka>

Функціональність системи

Клієнт взаємодіє з **facade-service** через HTTP POST- та GET-запити.

- Необхідно розгорнути **Messaging queue** - Hazelcast Distributed Queue чи Kafka
- При надходженні POST-запиту до **facade-service** він має додавати повідомлення з запиту до черги повідомлень для **counter-service**
- **counter-service** має вчитувати повідомлення (схема - producer/consumer) та змінювати відповідний баланс на рахунку

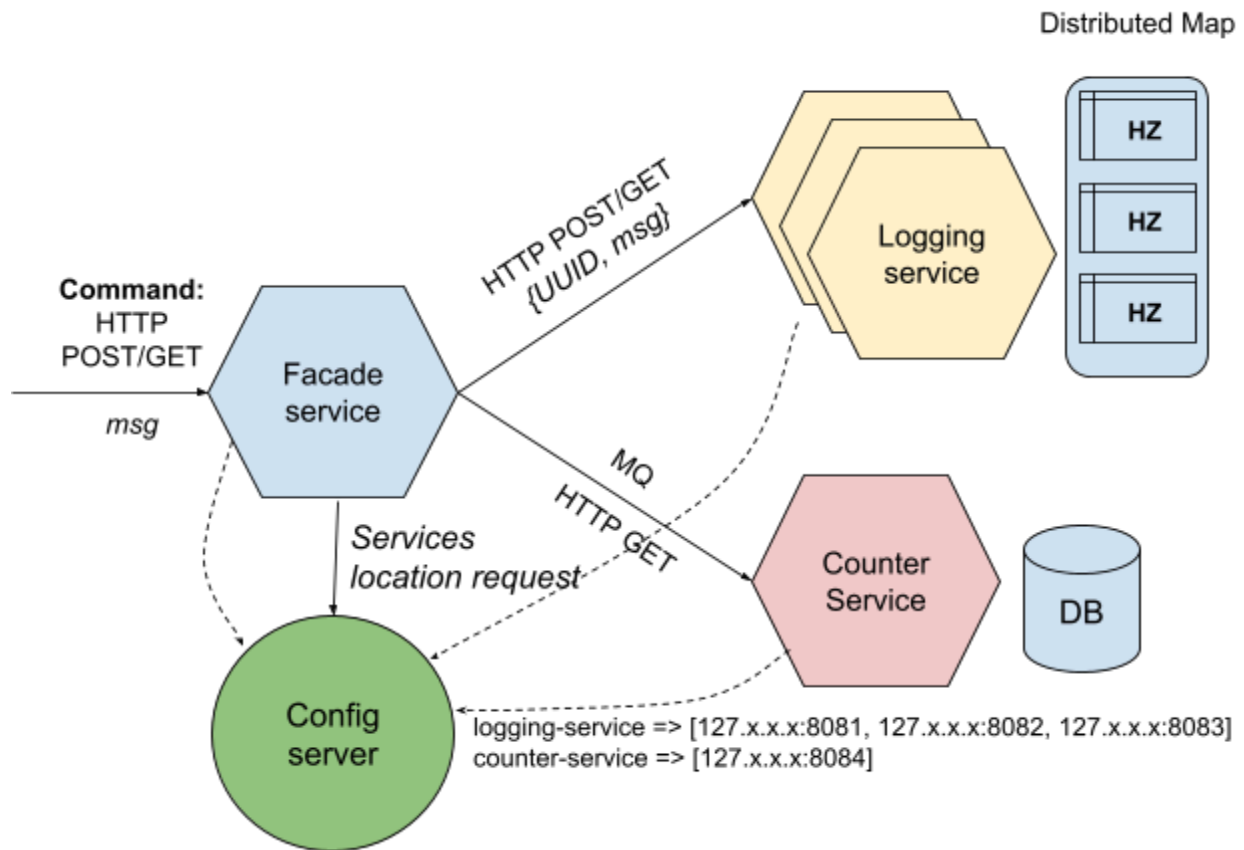


Configuration server

У зв'язку з тим, що тепер може бути декілька запущених екземплярів **logging-service**, **facade-service** має знати про їх IP-адреси для доступу до них. Перелік IP-адрес може міститися у коді самого **facade-service** чи передаватися у нього при старті, проте такий підхід не є гнучким у разі динамічного призначення чи зміни цих адрес.

Тому пропонується винести інформацію про IP-адреси інших мікросервісів у окремий реєстр, за який буде відповідати **config-server** (пізніше він буде замінений на *Service registry and discovery*).

Тепер, перед зверненням до **logging-service** та **counter-service**, **facade-service** має робити запит до **config-server**, де за ім'ям сервісу йому повертається перелік всіх IP-адрес екземплярів даного сервісу. Далі, випадковим чином обирається адреса до якої йде звернення, якщо існує декілька екземплярів мікросервіса



Звідки береться перелік IP-адрес мікросервісів на **config-server**?

Коли кожен з мікросервісів запускається, він має через POST-запит реєструвати себе на **config-server**.

Тобто можна припустити, що кожен мікросервіс має знати адресу **config-server**, яка має братися з конфігураційного файлу чи передаватися з командного рядка під час його запуску.

Завдання

- Запускається **config-server**,
- Запустити **facade-service**, **counter-service** та три екземпляра **logging-service** (кожен з екземплярів запускається у власному Docker-контейнері), і далі реєструється в **config-server**,
- Відповідно має запуститись також кластер Hazelcast з трьох серверів
- Через HTTP POST записати 10 транзакцій *msg1-msg10* через **facade-service**

- Показати що повідомлення отримують різні екземпляри **logging-service** (це має бути видно у логах сервісу)
- Через HTTP GET з **facade-service** прочитати транзакції, перевірити що значення на рахунках є коректним
- Перевірка відмовостійкості:
 - Вимкнути чи зробити недоступним (*docker pause*) **counter-service** та перевірити, що запис транзакцій йде без помилок (бо транзакції до **counter-service** накопичуються у message queue)
 - При цьому, відповідно GET-запити не мають спрацьовувати, мають або повертати null у якості значень на рахунку
 - Знову запустити **counter-service** та переконатись, що за деякий час він зчитує з черги накопичені до нього транзакції, застосує їх та почне повертати коректні значення рахунків

Вимоги до реалізації

Код всіх три мікросервісів має бути збережений у GitHub в одному репозиторії у гілці micro mq

Вимоги до протоколу

У протоколі має міститись:

- посилання на GitHub з проектом
- скріншоти з POST/GET-запитами та відповіді на них
- вміст консолі кожного з мікросервісу якщо туди виводиться якась корисна інформація